



Fundamentos da Programação Orientada a Objetos

UNIDADE 06

Classes abstratas e interfaces

| Abstraindo conceitos

Até aqui, você já estudou diversos mecanismos que a orientação de objetos fornece para que criemos abstrações (simplificações) do mundo real:

- Com as **classes**, descrevemos em forma de software os conceitos do mundo real que estão sendo implementados (ex.: usuários, conta bancária etc.).
- Agrupamos as classes por meio da relação de **associação**, entendendo suas interações ou situações do todo e suas partes.
- Com a **herança**, criamos relações de tipos entre as classes, agrupando-as em hierarquias, de acordo com sua similaridade.
- Com o **encapsulamento**, conseguimos esconder detalhes complexos da implementação, enquanto fornecemos uma **interface pública** de mais simples entendimento.
- Com os **pacotes**, agrupamos classes relacionadas por alguma característica ou função.
- Com o **sobrecarga** e o **polimorfismo** passamos a descrever o conceito de uma operação, mesmo que ela possa ser realizada de formas diferentes.

Nesta unidade, aprofundaremos ainda mais o conceito de abstração estudando dois novos temas. O primeiro são as **classes abstratas** e o segundo, **interfaces**. Ambos fornecem um mecanismo poderoso para generalizar ainda mais nossos códigos.

Também, aprenderemos sobre **Git** (pronunciamos como “guit”) nesta semana. Se você já pesquisou por algumas vagas na área você pode ter se deparado com este termo na lista de requisitos das vagas. Git é uma das ferramentas de controle de versão mais populares e amplamente utilizadas nas empresas. Controlar as versões de seu código permite que você acompanhe as mudanças, reverta para versões anteriores se necessário. Por isso, Git facilita a colaboração em projetos de software e permite que vários desenvolvedores trabalhem no mesmo projeto sem se atrapalharem, mesclando suas alterações.

Git se tornou um padrão da indústria. A maioria das empresas, pequenas ou grandes, o usa para controle de versão. É por isso que saber Git é frequentemente um requisito para cargos de desenvolvimento de software. Você poderá usar Git para manter seus códigos em Java, Python, ou qualquer outra linguagem de programação.

Vamos lá?



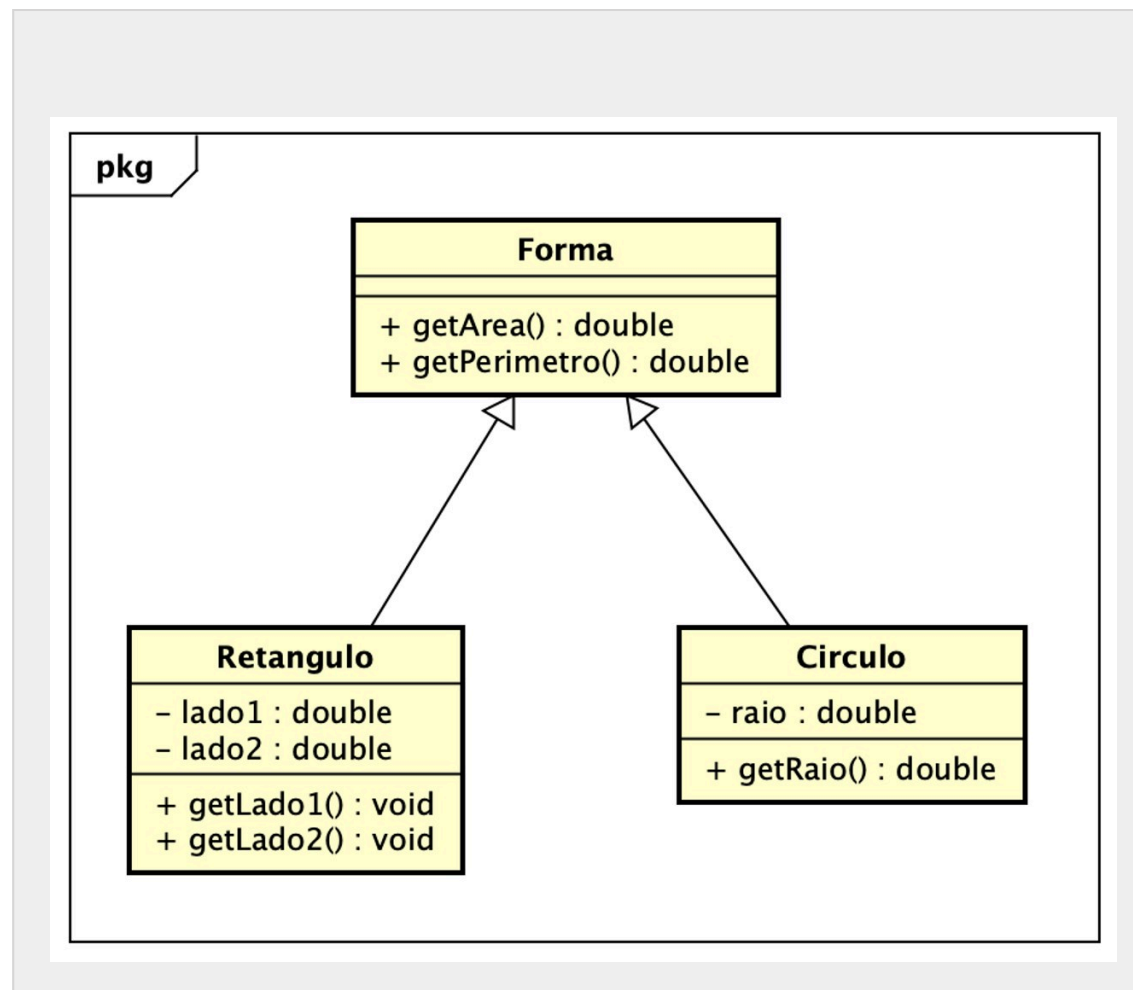
Classes abstratas

Primeiramente, vamos implementar um algoritmo do jeito que conhecemos – isto é, com classes e objetos. Vejamos:



EXEMPLO

Exemplo de aplicação 1: crie um projeto em Java a partir do diagrama UML a seguir:



Fonte: Autores (2021).

Tente implementar por você mesmo. Neste momento da disciplina você não deverá ter grandes dificuldades em fazer este código – se este for o caso, recomendo muito que faça os exercícios de fixação das semanas anteriores.

Bom, dito isso, se tiver alguma dificuldade, sugiro que compare o seu código com a nossa implementação, a seguir:

Arquivo Forma.java

</>

```
1 public class Forma {  
2     public double getArea() {  
3         return 0;  
4     }  
5  
6     public double getPerimetro() {  
7         return 0;  
8     }  
9 }
```

Arquivo Retangulo.java

</>

```
1 public class Retangulo extends Forma {
2     private double lado1;
3     private double lado2;
4
5     public Retangulo(double lado1, double lado2) {
6         this.lado1 = lado1;
7         this.lado2 = lado2;
8     }
9
10    public double getLado1() {
11        return lado1;
12    }
13
14    public double getLado2() {
15        return lado2;
16    }
17
18    @Override
19    public double getArea() {
```

Arquivo Circulo.java

</>

```
1 public class Circulo extends Forma {
2     private static final double PI = 3.1415;
3     private double raio;
4
5     public Circulo(double raio) {
6         this.raio = raio;
7     }
8
9     public double getRaio() {
10         return raio;
11     }
12
13     @Override
14     public double getArea() {
15         return PI * raio * raio;
16     }
17
18     public double getPerimetro() {
19         return 2 * PI * raio;
```

Arquivo Main.java

</>

```
1 import java.util.*;
2
3 public class Main {
4     public static void imprimir(List<Forma> formas) {
5         for (Forma forma : formas) {
6             System.out.printf("Area: %.2f Perimetro: %.2f%n",
7                 forma.getArea(), forma.getPerimetro());
8         }
9     }
10
11     public static void main(String[] args) {
12         List<Forma> formas = new ArrayList<Forma>();
13         formas.add(new Retangulo(2, 2));
14         formas.add(new Circulo(3));
15         formas.add(new Retangulo(4, 3));
16         formas.add(new Circulo(1.5));
17         //formas.add(new Forma());
18         imprimir(formas);
19     }
```

Se o seu código estiver funcionando como esperado, você deverá ser capaz de ver um resultado assim a partir dos testes que fizemos nas linhas 12 a 17 no arquivo **Main.java**:

```
Area: 4.00 Perimetro: 8.00
Area: 28.27 Perimetro: 18.85
Area: 12.00 Perimetro: 14.00
Area: 7.07 Perimetro: 9.42
```

Process finished with exit code 0

Agora, você não teve aquela sensação de que estava fazendo um “ajuste técnico” (ou, “gambiarra”, mesmo) no código? Vejamos:

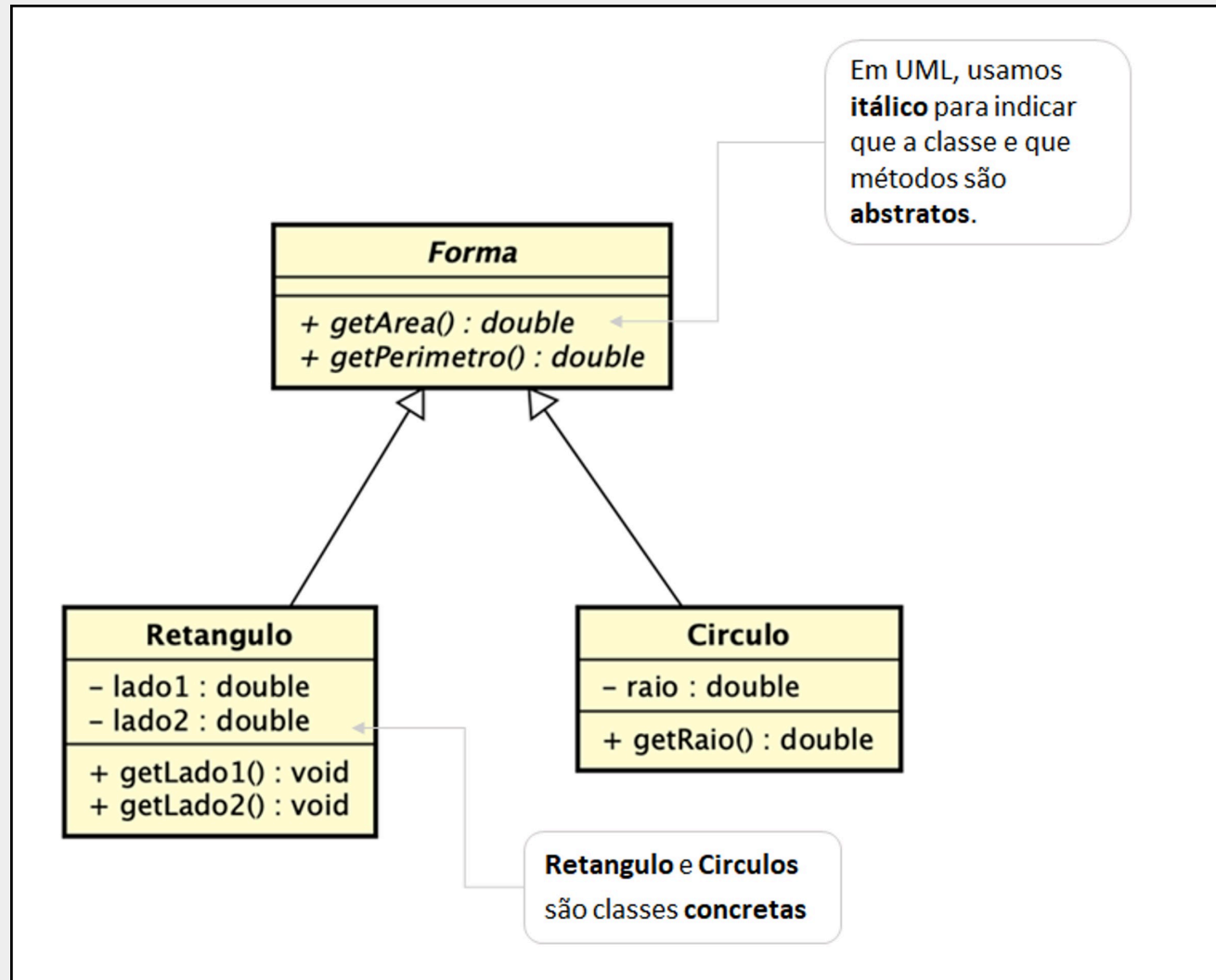
1. Em **Forma.java**, fomos obrigados a retornar 0 nos métodos **getArea()** e **getPerimetro()** (linhas 3 e 7). Por que tivemos de colocar 0? Esse valor não faz sentido algum, mesmo assim, fomos obrigados a colocar algum valor para que o código não desse um erro.
2. E se tentássemos instanciar uma forma diretamente? Quer dizer, e se removêssemos o comentário (as duas barras) no início da linha 17 do arquivo **Main.java** para que pudéssemos testar uma instância de **Forma**? Isso a princípio não faria sentido. Pensemos: como calcular a área e um perímetro de uma área em que nem sabemos o formato? E se essa forma não fosse nem um círculo e nem um retângulo?

O exercício traz um exemplo claro da nossa capacidade de **abstração**. Perceba que, enquanto quadrado e círculo são conceitos concretos (e podemos saber exatamente ao que se referem), quando criamos a superclasse os agrupamos em um conceito **abstrato**: o da forma. E o que queremos dizer por abstrato? Sabemos que todas as formas têm uma área e um perímetro, mas exatamente como calculá-los só será possível quando estivermos diante de uma forma específica.

Agora, pense um pouco. E se alguém falasse para você: “desenhe uma forma no caderno”, o que você desenharia? Provavelmente, perguntaria “qual forma você quer que eu desenhe?”. Portanto, até podemos nos referenciar a círculos ou quadrados como formas, mas somente objetos concretos existem, e não uma “forma genérica”.

E como podemos representar essa abstração na orientação a objetos? Como implementá-la em Java? Fazemos isso por meio do conceito de **classe abstrata**. Como representa um conceito abstrato, uma classe desse tipo não poderá ser instanciada com a palavra-chave **new**. Além disso, ela pode conter **métodos abstratos**, sem qualquer implementação. Ou seja, indicaremos na classe que sabemos que aquela operação existe para todos os objetos daquele tipo, mas somente as instâncias concretas irão implementá-lo, por meio da sobreposição.

Vamos então reescrever a classe **forma**? Primeiramente, veja como fica o diagrama UML modificado para conter a classe abstrata:



Fonte: Autores (2021).

Agora, veja como fica o código modificado de Forma.java:

</>

```
1 public abstract class Forma {  
2     public abstract double getArea();  
3     public abstract double getPerimetro();  
4 }
```

Viu as palavras **abstract**? Além disso, não precisamos mais implementar nada nas funções. Vamos às diferenças básicas entre classes abstratas e classes concretas em Java:

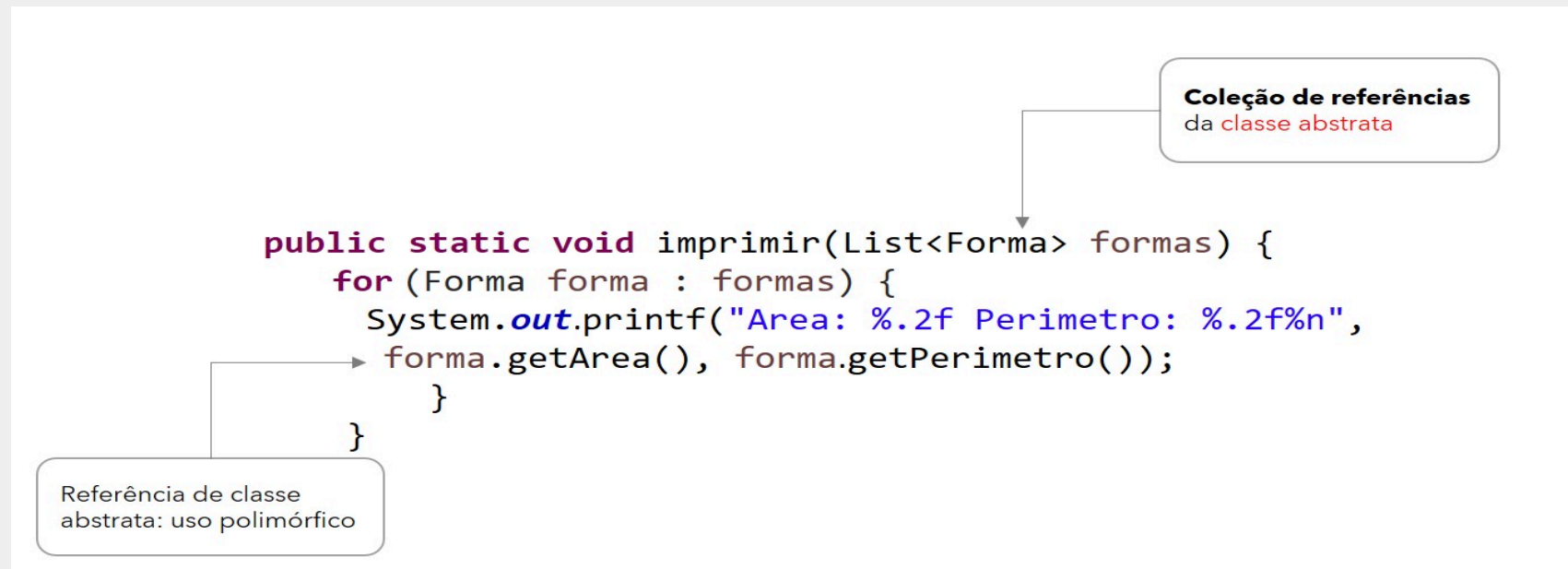
1. Classes abstratas:

1. Uma classe abstrata em Java é declarada usando a palavra-chave **abstract**.
2. Ela pode incluir métodos abstratos (métodos sem implementação de uma lógica) e métodos concretos (métodos com implementação de uma lógica). O exemplo do código modificado da classe **forma** possui dois métodos abstratos (sem implementação de lógica).
3. As classes abstratas **não podem ser instanciadas**. Ou seja, você não pode criar um objeto a partir de uma classe abstrata **diretamente**. A sua principal finalidade é ser estendida (herdada) por outras classes.

2. Classes concretas:

1. Ao contrário das classes abstratas, as classes concretas em Java são as classes regulares que você usa para criar objetos.
2. Elas podem incluir métodos e variáveis, e todos os métodos devem ter um corpo de método.
3. As classes concretas são o "núcleo" do seu programa, onde a maior parte da lógica de negócios está contida.

Qualquer classe concreta que implemente de forma será **obrigada** a sobrescrever **todos** os métodos abstratos, no caso, `getArea()` e `getPerimetro()`. Assim, podemos considerar a classe `forma` como um **modelo para a criação de novas classes**. Esses métodos poderão ser usados por meio de polimorfismo, tal como fizemos no método de impressão na classe `main`. Vamos nos lembrar da implementação do método **imprimir()** (arquivo `Main.java`, linhas 4 a 9):



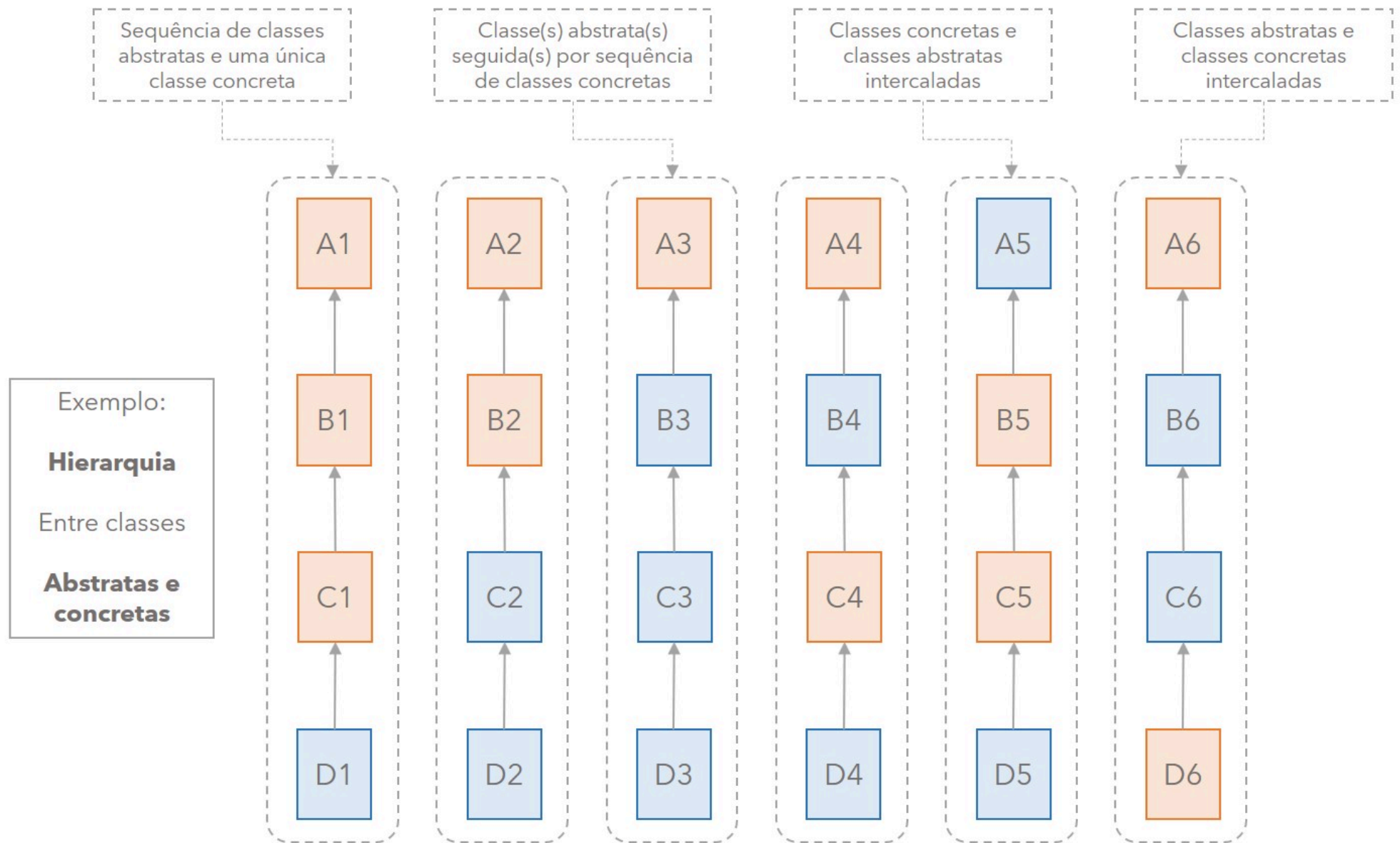
Fonte: Autores (2021)

Um ponto importante a entender é que você pode ter uma classe abstrata que é derivada (ou filha) de outra classe, que pode ser tanto abstrata quanto não abstrata. Se isso acontecer, você tem a opção de adicionar mais métodos abstratos na classe-filha. Além disso, não é obrigatório implementar todos os métodos abstratos que foram herdados da classe-pai. Esses métodos podem continuar sendo abstratos na classe-filha.

Vamos pensar em analogias para que isso fique mais claro? Pensemos em uma classe chamada **animal**. Animal é um conceito abstrato, não é? Quer dizer: eu posso estar pensando em um cachorro, e você em um tubarão. Ambos são animais, mas bem diferentes um do outro. Agora, podemos ter uma outra classe chamada **mamifero** que herda de **animal**. Esse conceito **continua sendo abstrato**: mamíferos têm as suas próprias particularidades, mas um humano é bem diferente de um elefante, por exemplo. Finalmente, a classe **elefante** pode herdar de **mamifero**. O elefante é concreto. Da mesma forma, **ElefanteAsiatico** pode herdar de **elefante**. Ele continua sendo um elefante, mas é uma especialização dele.

O diagrama a seguir resume todas as opções nesse sentido – o exemplo do Animal (A2)/Mamifero (B2)/Elefante (C2)/ElefanteAsiatico (D2) corresponde à segunda coluna do diagrama. Aqui, temos uma sequência de duas classes abstratas (na cor rosa) e duas concretas (na cor azul). Perceba que também, podemos ter outros tipos de combinações de classes concretas e abstratas.

#ParaTodosVerem 





IMPORTANTE

Não existe obrigatoriedade de que uma classe abstrata tenha métodos abstratos. Você pode implementar métodos concretos em uma classe abstrata sem problemas.

Vamos para mais um exercício? Agora, será um pouco diferente: vamos analisar um código já existente e procurar responder corretamente às perguntas. Você não precisará entregar as respostas em lugar algum: responda por si mesmo para ver se entendeu bem os conceitos.



DICA

Imagine que quem está fazendo essas perguntas a você é uma analista sênior em uma entrevista de emprego. Logo, ela procurará identificar falhas ou inconsistências na sua resposta para ver se você conhece estes conceitos ou não. Portanto, este é o momento para realmente entender estes conceitos – na dúvida, implemente códigos com uma funcionalidade parecida antes de responder algo sem ter certeza (em outras palavras: “no chute”).

Vamos lá:



EXERCÍCIO

Exemplo de aplicação 2: fixação de conceitos de classes abstratas.

1. Pare para pensar um pouco e responda: um método abstrato pode ser declarado como private?
2. Considere as classes declaradas a seguir:

```
abstract class Pessoa {...}
class Estudante extends Pessoa { ... }
class Professor extends Pessoa { ... }
class EstudanteEspecial extends Estudante { ... }
```

Quais dessas instanciações são possíveis? Como você chegou a essa conclusão?

1. `Pessoa p1 = new Professor();`
2. `Professor prof = new Professor();`
3. `Professor prof2 = new Estudante();`
4. `Pessoa p2 = new Pessoa();`
5. `Pessoa p3 = new EstudanteEspecial();`
6. `Estudante e = new EstudanteEspecial();`

Os códigos a seguir estão corretos? Justifique sua resposta.

a)

```
abstract class Impressora {
    protected void print() {
        System.out.println(this);
    }
}
```

b)

```
abstract class Vazia {}
```

c)

```
class JogadaDeDados {  
    private int dados;  
    private int faces;  
  
    public abstract int jogar();  
}
```

d)

```
abstract class Animal {  
    protected String nome;  
  
    public abstract void fazerSom();  
    protected abstract void respirar();  
    public abstract void imprimir() {  
        System.out.println(nome);  
    }  
}
```



Resolução do exercício



Resposta 1 – Instanciações possíveis:

a) Possível: professor é uma classe concreta e filha de pessoa:

Professor **instanceof** Pessoa == true

b) Possível: instanciação de classe concreta

c) Não é possível: professor e estudante no mesmo nível da hierarquia. Um estudante não é um professor, nem vice-versa.

d) Não é possível: pessoa é uma classe abstrata, não pode ser instanciada.

e) Possível: EstudanteEspecial é um filho indireto de pessoa: EstudanteEspecial instanceof Pessoa == true

f) Possível: EstudanteEspecial é uma classe filha de pessoa: EstudanteEspecial instanceof Estudante

Resposta 2 – Declarações:

a) Correta: uma classe abstrata não precisa conter métodos abstratos.

b) Correta: qualquer classe pode ou não ter métodos ou atributos. Uma classe vazia é perfeitamente válida, abstrata ou não!

c) Incorreta: a classe JogadaDeDados não é abstrata, portanto, o método jogar() não pode ser abstrato. Ele deveria ser implementado, ou a classe deveria ser declarada também como abstrata.

c) Não é possível: professor e estudante estão no mesmo nível da hierarquia. Um estudante não é um professor, e nem vice-versa.

d) Correta: uma classe abstrata pode ter métodos e atributos não abstratos, com qualquer visibilidade. Também podem ter métodos abstratos, desde que sua visibilidade não seja private.

e) Possível: EstudanteEspecial é um filho indireto de pessoa:

`EstudanteEspecial instanceof Pessoa == true`

f) Possível: EstudanteEspecial é uma classe filha de pessoa:

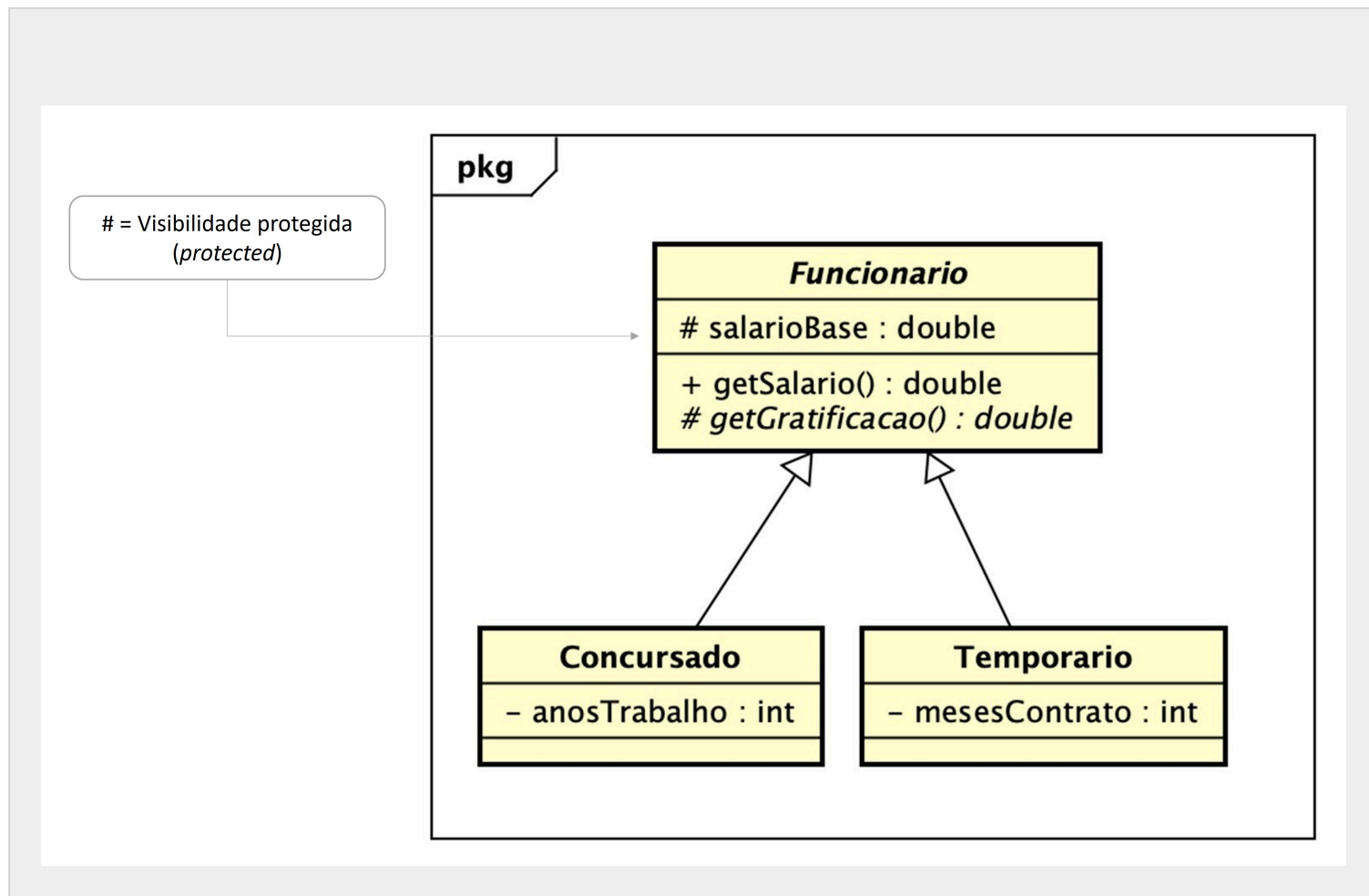
`EstudanteEspecial instanceof Estudante`



EXEMPLO

Exemplo de aplicação 3: implemente o código a partir do diagrama UML a seguir.

#ParaTodosVerem 



Regras:

1. O cálculo do salário é dado por:

Salário-base + gratificação.

2. A gratificação dos funcionários concursados é:

2% do salário-base a cada cinco anos completos de casa (quinqüênio).

3. A gratificação dos funcionários temporários é:

R\$ 100,00 para cada mês de contrato.

4. Crie construtores nas classes de modo que seja possível informar o valor de todos os atributos. Ex.:

```
var joao = new Concursado(4000, 5);
```

1. Crie uma classe **main** com uma lista de funcionários. Em seguida, crie um método de impressão para listar todos os funcionários:

1. João, concursado, com salário-base de R\$ 4000,00 e 5 anos de profissão.
2. José, temporário, com salário de R\$ 2000,00 e 12 meses de contrato.
3. Maria, concursada, com salário mensal de R\$ 2400,00 e 34 anos de casa.
4. Pedro, temporário, com salário mensal de R\$ 1200,00 e 10 meses de contrato.

Como de costume, procure implementar este exercício por conta própria. Dito isso, vamos entender como seria uma resposta válida a partir dos exemplos a seguir.

Arquivo Funcionario.java

```
</>
```

```
1 public abstract class Funcionario {
2     protected double salarioBase;
3
4     public double getSalario() {
5         return this.salarioBase + getGratificacao();
6     }
7
8     protected abstract double getGratificacao();
9 }
```

Este código define uma classe abstrata chamada **funcionario**. Isso significa que não podemos instanciar um objeto da classe **funcionario** diretamente. Em vez disso, devemos estender a classe **funcionario** e implementar quaisquer métodos abstratos que ela declare.

O código contém o seguinte:

- Um atributo **protected** chamado **salarioBase**. Lembre-se de que o **protected** significa que o atributo só pode ser acessado diretamente pela própria classe e por qualquer classe que a estenda. Isso é útil quando queremos que classes-filhas tenham acesso direto a algum atributo ou método, mas queremos escondê-lo de outras partes do código.
- Um método **public** chamado **getSalario()**, que retorna a soma do **salarioBase** com uma **gratificacao**. Observe que **getGratificacao()** é um método abstrato, então o método **getSalario()** está usando um método que não foi definido ainda e que precisaremos implementar em todas as classes que herdem de **funcionario**.
- O método **protected abstract double getGratificacao()**. Isso é uma declaração de um método abstrato. Um método abstrato é um método sem corpo – apenas a assinatura é fornecida. Isso sinaliza que qualquer classe que estenda **funcionario** deve fornecer uma implementação para este método.

Ah, e veja só: o método **getSalario()** está fazendo uso do polimorfismo. A ideia é que diferentes tipos de **funcionario** podem ter maneiras diferentes de calcular a **gratificacao**, mas todos eles terão algum tipo de **gratificacao** que será adicionada ao **salarioBase** para obter o salário total. Portanto, qualquer classe que estenda **funcionario** terá de implementar o método **getGratificacao()**, definindo como a **gratificacao** é calculada para esse tipo específico de **funcionario**.

</>

```
1 public class Temporario extends Funcionario {
2     public int mesesContrato;
3
4     public Temporario(double salarioBase, int mesesContrato){
5         this.salarioBase = salarioBase;
6         this.mesesContrato = mesesContrato;
7     }
8
9     protected double getGratificacao(){
10         return this.mesesContrato * 100;
11     }
12 }
```

Temos aqui uma classe **temporario** que estende a classe abstrata **funcionario**. Isso significa que **temporario** é um tipo específico de **funcionario**, com características e comportamentos adicionais específicos para funcionários temporários.

Vejamos alguns pontos sobre este código:

- A classe **temporario** possui um atributo **public** chamado **mesesContrato**. Isso representa a quantidade de meses que o contrato do funcionário temporário dura.
- O construtor da classe **temporario** aceita dois parâmetros: **salarioBase** e **mesesContrato**, e usa esses valores para inicializar os atributos correspondentes da classe. O atributo **salarioBase** é herdado da classe **funcionario**.
- O método **getGratificacao()** é a implementação do método abstrato que foi declarado na classe **funcionario**. Ele calcula a gratificação do funcionário temporário com base nos meses de contrato. Cada mês de contrato rende ao funcionário uma gratificação de 100 unidades monetárias (o código não especifica a moeda, mas sabemos que se trata de R\$).

O `getGratificacao()` na classe **temporario** é um exemplo de polimorfismo. Embora a classe **temporario** seja uma subclasse da classe **funcionario**, ela fornece uma implementação específica do método `getGratificacao()`, adequada para funcionários temporários. Outros tipos de **funcionario** podem ter diferentes implementações deste método.

Arquivo Concursado.java

</>

```
1 public class Concursado extends Funcionario {
2     public int anosTrabalho;
3
4     public Concursado(double salarioBase, int anosTrabalho){
5         this.salarioBase = salarioBase;
6         this.anosTrabalho = anosTrabalho;
7     }
8
9     protected double getGratificacao(){
10         int quinquenio = this.anosTrabalho / 5;
11
12         return quinquenio * 0.02 * this.salarioBase;
13     }
14 }
```

Aqui, definimos uma classe **concurso** que estende a classe abstrata **funcionario**. Isso significa que **concurso** é um tipo específico de **funcionario**, com certas características e comportamentos adicionais.

Vejamos:

- A classe **concurso** possui um atributo **public** chamado **anosTrabalho**. Isso representa a quantidade de anos que o funcionário concursado trabalhou.

- O construtor de **concursado** aceita dois parâmetros, **salarioBase** e **anosTrabalho**, e os usa para inicializar os atributos correspondentes da classe. O **salarioBase** é um atributo herdado da classe **funcionario**. Por isso, não tivemos de recriá-lo aqui.
- O método **getGratificacao()** é a implementação do método abstrato que foi declarado na classe **funcionario**. Ele calcula a gratificação com base nos anos de trabalho do funcionário. Para cada quinquênio (5 anos), o funcionário recebe uma gratificação equivalente a 2% do seu salário base. A variável **quinquenio** é calculada dividindo **anosTrabalho** por 5.

Observe que o método **getGratificacao()** na classe **concursado** é um outro exemplo de polimorfismo (assim como já vimos em **temporario**). A classe **concursado** fornece sua própria implementação para o método **getGratificacao()** que é específico para concursados, mas veja que **temporario** podia (e teve) uma implementação diferente para **getGratificacao()**.

Arquivo Main.java

</>

```
1  import java.util.*;
2
3  public class Main {
4      public static void imprimir(List<Funcionario> funcionarios) {
5          for (Funcionario funcionario : funcionarios) {
6              System.out.printf("Salário base: %.2f Salário com gratificação: %.2f%n",
7                  funcionario.salarioBase, funcionario.getSalario());
8          }
9      }
10
11     public static void main(String[] args) {
12         var funcionarios = new ArrayList<Funcionario>();
13         funcionarios.add(new Concursado(4000, 5));
14         funcionarios.add(new Temporario(2000, 12));
15         funcionarios.add(new Concursado(2400, 34));
16         funcionarios.add(new Temporario(1200, 10));
17         imprimir(funcionarios);
18     }
19 }
```

Este código é bem parecido com as implementações que tivemos em exemplos anteriores com o **main**. O método **imprimir()** recebe uma lista de objetos do tipo **funcionario**. Para cada **funcionario** na lista, ele imprime o salário-base e o salário com gratificação.

Note que, embora a lista seja do tipo **funcionario**, você pode adicioná-la objetos das classes **concursado** e **temporario**, pois essas duas classes estendem **funcionario**. Isso é um exemplo de polimorfismo: você pode tratar objetos de **concursado** e **temporario** como objetos de **funcionario**.

O método **main()**, como sempre, é o ponto de entrada do programa. Ele cria uma lista **ArrayList** de **funcionario**, adiciona nela alguns objetos **concursado** e **temporario**, e então chama o método **imprimir()** passando essa lista. Quando você executa este código, o método **getSalario()** chamado para cada **funcionario** na lista será a versão do método que corresponde ao tipo real do objeto (**concursado** ou **temporario**), não o tipo da variável (**funcionario**). Isso é chamado de **vinculação dinâmica**, e é outra manifestação de polimorfismo.

Em termos de saída, o programa imprimirá o salário base e o salário com gratificação para cada **funcionario** na lista, usando duas casas decimais para os valores monetários.

| Interfaces

Como você estudou, a relação de herança produz uma relação de tipos entre a classe-pai e a filha. Todos os métodos e atributos são herdados. Não é à toa que dizemos que a herança cria uma **relação forte**, pois, como as classes, herdam os atributos e implementações. Isso tem uma implicação importante: uma modificação na classe-pai poderá impactar em todas as suas classes-filhas – sendo o ideal testar todas as classes para ver se um erro inesperado não foi acidentalmente inserido.

Há, porém, uma alternativa: as interfaces. Elas são similares às classes abstratas, porém com algumas limitações:

1. Não podem ter atributos de estado (variáveis de instância), mas apenas constantes.
2. Somente podem conter métodos públicos e abstratos.
3. Uma interface pode fazer herança, mas somente com outras interfaces. Porém, a limitação de uma única interface pai não existe.

Se são parecidos com as classes abstratas, por que existem? Vejamos:

Na escolha entre interfaces e classes abstratas em Java, os seguintes critérios podem ser considerados:

1. **Quantidade de implementação compartilhada:** Se várias classes devem compartilhar uma quantidade considerável de código, pode ser mais apropriado usar uma **classe abstrata**. As classes abstratas permitem que você defina métodos com comportamento padrão, além de métodos abstratos que as subclasses são obrigadas a implementar. As interfaces, por outro lado, não permitiam a definição de métodos

com implementação até o Java 8. Depois do Java 8, as interfaces podem ter métodos default e métodos estáticos com implementação, mas não têm variáveis de estado, então não podem ser usadas para compartilhar dados de estado entre classes.

- Existem empresas que por questões de retrocompatibilidade de código ainda não podem usar o Java 8. Logo, ainda que ele exista há uma década, nem sempre você trabalhará com o Java 8 ou outras versões mais atuais.

2. **Relação "é um" versus "tem um"**: se a relação entre as classes for do tipo "é um" (por exemplo, um **cão** é um **animal**), pode ser mais apropriado usar classes abstratas. Se a relação for do tipo "tem um" (por exemplo, um **cão** tem um comportamento **caçador**), pode ser mais apropriado usar interfaces.
3. **Herança múltipla**: em Java, uma classe pode estender **apenas uma** classe, mas pode implementar **várias** interfaces. Se você precisa que uma classe herde comportamentos de várias fontes, você pode precisar usar interfaces.
4. **Flexibilidade do design**: as interfaces são geralmente consideradas **mais flexíveis** do que as classes abstratas, porque permitem a separação de comportamentos. Isso pode tornar o design de software mais modular e fácil de alterar ou estender ao longo do tempo.
5. **Versões futuras do software**: se você está criando uma interface de aplicações (API) que será usada por outras pessoas, e acredita que essa API pode mudar no futuro, geralmente é melhor usar interfaces. Se você adicionar novos métodos a uma interface, você pode definir implementações padrão para esses métodos que não quebrarão o código existente. Se você adicionar novos métodos a uma classe abstrata, qualquer classe que estende essa classe abstrata sem implementar os novos métodos será quebrada.

Criamos as interfaces por meio da palavra-chave **interface**. E declaramos interfaces uma classe que implementa por meio da palavra-chave **implements**. Uma classe pode implementar qualquer número de interfaces, e isso pode ocorrer em qualquer nível da hierarquia de classes.

Outras similaridades ainda são observadas:

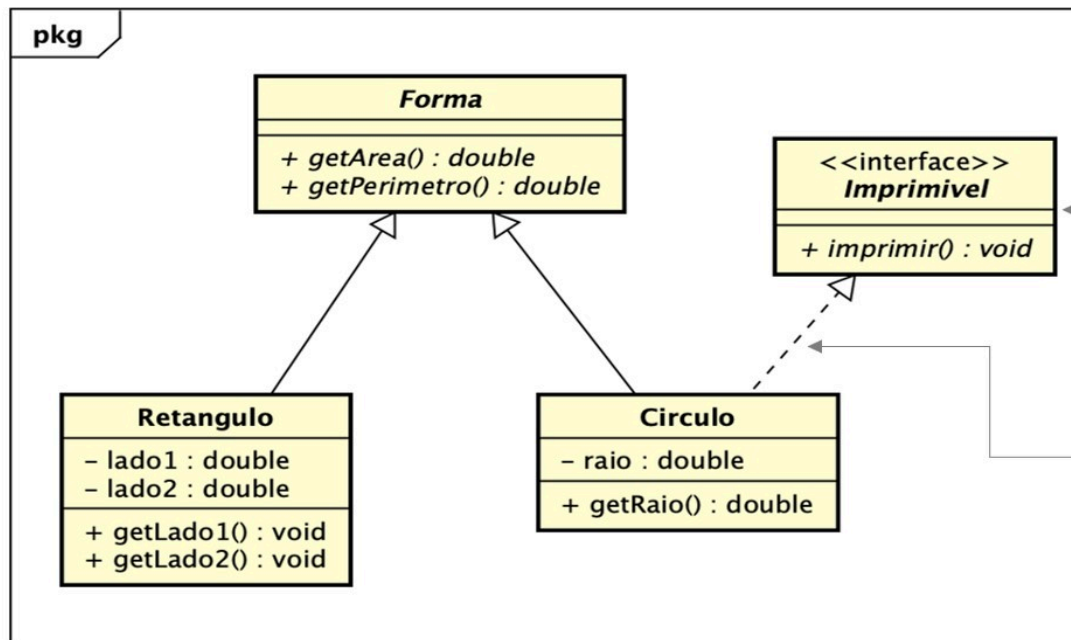
1. O operador **instanceof** funcionará para interfaces igualmente como funciona para a herança.
2. Todos os métodos abstratos das interfaces também terão de ser sobrepostos nas classes concretas.

Vamos testar um exemplo disso?



EXEMPLO

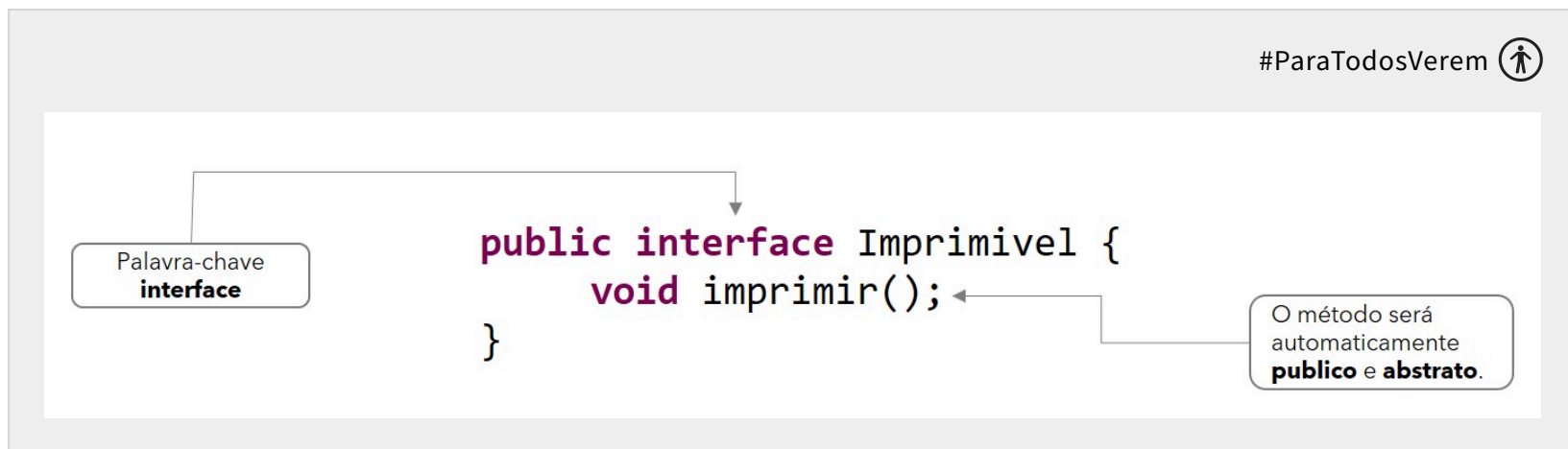
Exemplo de aplicação 4: analise o código proposto a partir do diagrama UML a seguir.



A notação é igual a de uma classe abstrata, com a adição de um **estereótipo** a interface

A relação de **realização** indica que a interface será implementada

Declaração



Fonte: Autores (2021).

Uso

```
@Override
public class Circulo extends Forma implements Imprimivel {
    ...

    @Override
    public void imprimir() {
        System.out.printf("Circulo - Area: %.2f%n", getArea());
    }
}
```

Palavra-chave **implements**

Implementação do método imprimir()

Fonte: Autores (2021).

Conceitualmente, podemos imaginar uma interface como um **contrato** entre a classe e o mundo exterior. A interface define os termos desse contrato, descrevendo qual é o comportamento esperado por aquela classe, enquanto a classe que o implementa compromete-se a cumprir esse contrato.

Como não há compromisso com uma implementação, a relação entre a interface e as classes que a implementam é bem mais fraca do que a herança. Sempre que puder, dê preferência ao uso de interfaces no lugar de classes abstratas.

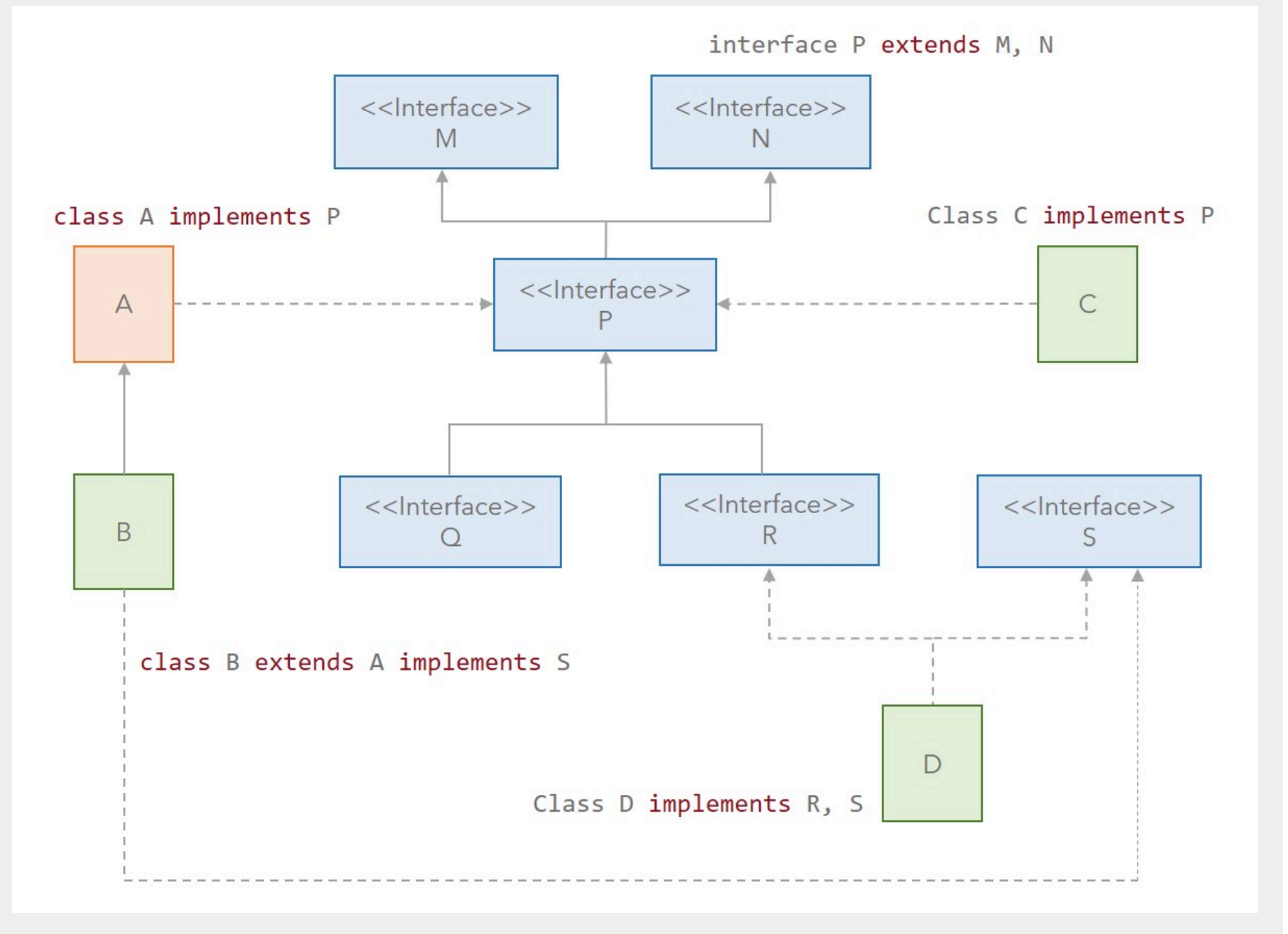


EXERCÍCIO



Exemplo de aplicação 5: Considere a hierarquia de classes e interfaces a seguir. Para melhorar a visualização, classes abstratas foram destacadas em laranja (classe A), concretas em verde (classes B, C e D) e as interfaces em azul (interfaces M, N, P, Q, R e S).

#ParaTodosVerem 



Após analisar, quais das alternativas a seguir estão corretas? Procure pensar nas respostas com um embasamento e a partir do seu entendimento, e não a partir de uma inferência ou adivinhação (ou seja, sem “chute”):

- a. **B instanceof P** retornará verdadeiro.
- b. Se C tentasse implementar R, daria erro, uma vez que haveria métodos redundantes entre P e R.
- c. A interface P está incorreta, já que você pode dar **implements** em várias interfaces, mas a herança com **extends** só permite um pai.
- d. Se um método aceitar um parâmetro do tipo M, poderemos utilizar objetos de qualquer classe descrita no diagrama.



Resolução do exercício



Resposta – Instanciações possíveis:

- a) Correta: como B é filho de A e A implementa a interface P, então B também implementará a interface indiretamente.
- b) Incorreta: como os métodos de interfaces não contêm implementação, não há conflitos em casos de redundâncias.
- c) Incorreta: uma interface pode ter múltiplas interfaces pai. Apenas classes estão limitadas a estender de um único pai.
- d) Correta: todas as classes possuem implementação, direta ou indiretamente, com a interface M.

Você deve ter percebido um padrão infeliz: os códigos que você escreve estão ficando cada vez mais longos e difíceis de serem mantidos! Antes, eram programas simples, com um único arquivo. Agora, com as classes, o código já se divide em vários arquivos – um por classe – em várias pastas, uma por pacote.

Além disso, à medida que o sistema evolui, criamos funções, movemos trechos de código de lugar, apagamos trechos de código, mudamos regras etc.

Naturalmente, vem o “medo” dessas mudanças. E se uma função que foi apagada voltar a ser necessária? E se um erro for introduzido em uma modificação e precisar voltar atrás? Quem nunca se sentiu tentado a criar um backup em um determinado momento para “garantir” que aquela versão do código sempre exista? Esse problema se torna ainda pior se você se lembrar que, profissionalmente, você raramente programará sozinho.

Como gerenciar o código de uma equipe inteira? Já imaginou se tivéssemos 200 pessoas trabalhando **no mesmo projeto de código ao mesmo tempo**?

Felizmente, uma solução profissional para esse problema já foi criada. São os **sistemas de controle de versão**. Eles permitem a criação de **repositórios de código**, em que as alterações são controladas.

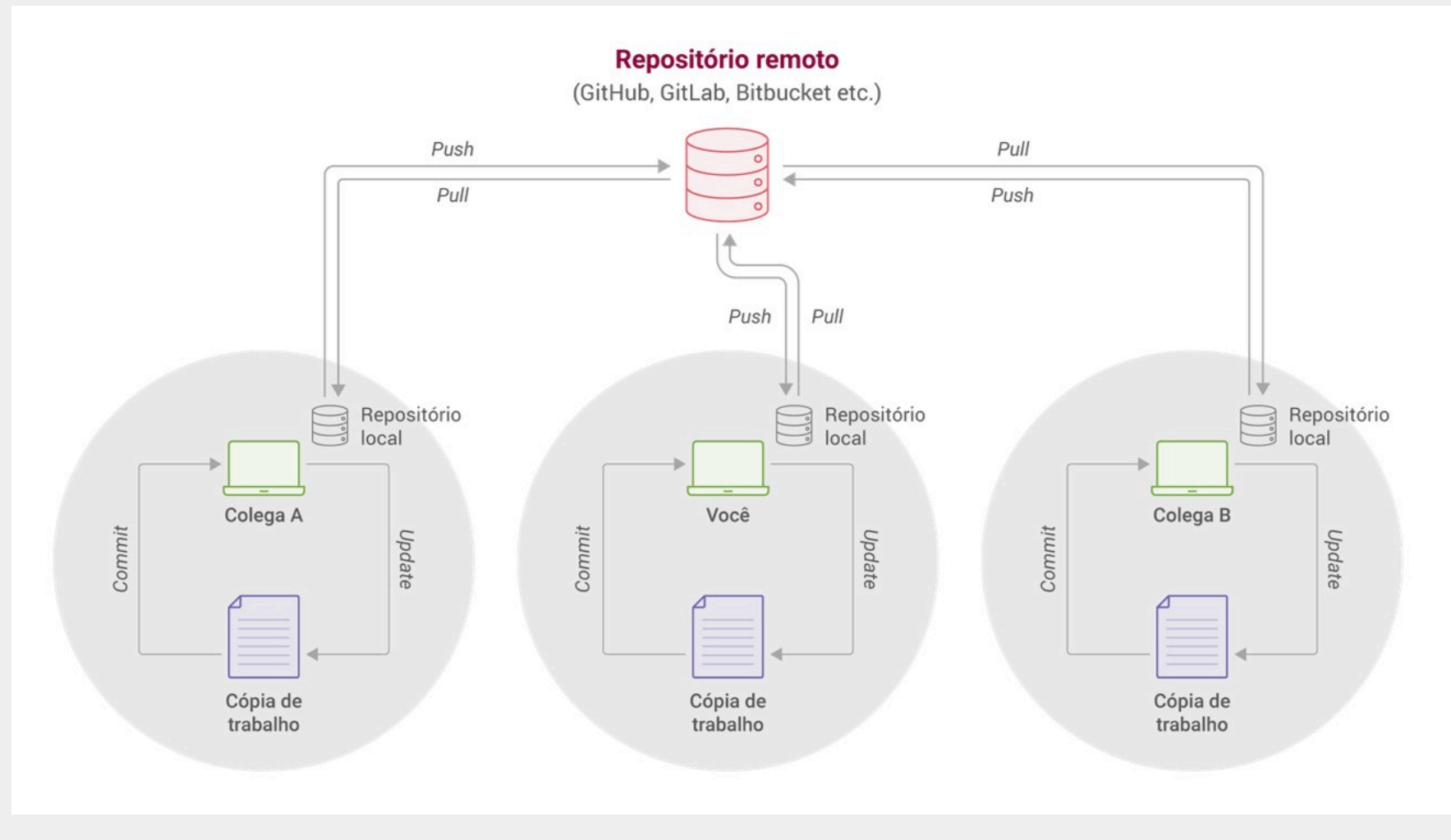


IMPORTANTE

Os **sistemas de controle de versão** permitem verificar histórico dos arquivos, gerenciar conflitos do código criado em equipe, marcar o código quando uma versão do software for produzida, compartilhar o código na internet, rastrear bugs, colabore com outras pessoas ao mesmo tempo etc.

| O sistema de controle de versão Git

Um dos sistemas de controle de versão mais populares e usados pelas empresas é chamado de **Git** e foi desenvolvido por Linus Torvalds, o mesmo que também criou o sistema operacional Linux. Além de gratuito, o Git possui uma arquitetura distribuída, permitindo que você sincronize um repositório com outro. Assim, você poderá trabalhar em uma versão do código em sua máquina, enquanto seus colegas fazem o mesmo nas deles, utilizando uma versão local do repositório – com todo conforto que o controle de versão permite. Quando as modificações estão prontas, você poderá enviar todo trabalho para um repositório remoto, normalmente compartilhado entre os membros da equipe. Essa arquitetura é demonstrada na figura a seguir.



Fonte: Autores (2021).

Outra vantagem é que o repositório compartilhado pode estar na internet, e há diversos serviços gratuitos que permitem o compartilhamento – público ou não – de seu código. Isso garantirá não só que você possa trabalhar em equipe, como também já fará o backup do seu programa de forma segura.

| Como começar a trabalhar com Git?

Felizmente, começar a usar Git não é tão complicado quanto possa parecer. Existem várias plataformas de hospedagem de repositórios Git, como GitHub, GitLab e Bitbucket. Estes sites permitem que você armazene seu código online, compartilhe-o com outras pessoas e colabore em projetos. Dentre estes, o GitHub é o mais conhecido e o mais popular.



IMPORTANTE

Que Git e GitHub não são a mesma coisa. Git é o sistema de controle de versão que mantém o registro das alterações em seu código. Já o GitHub é um serviço online que usa Git para ajudá-lo a compartilhar e colaborar no seu código com outros. Ou seja, você usa o Git para controlar as versões do seu código e o GitHub para hospedá-lo na internet. O mesmo se aplica aos seus concorrentes, como GitLab e Bitbucket.

Assim, existem algumas opções para você – escolha uma:

1. **GitHub:** se você quiser usar o GitHub, crie primeiro a sua conta em <https://github.com>. Depois, faça opcionalmente o [tutorial de uso do próprio Github \(em português\)](#) e o [tutorial de Git \(em português\)](#).
2. **Bitbucket:** se você quiser usar o Bitbucket, crie a sua conta em <https://www.atlassian.com/br/software/bitbucket/bundle>. Depois, faça opcionalmente o [tutorial de uso do BitBucket \(em português\)](#).
3. **GitLab:** você também pode optar em usar o GitLab. Crie a sua conta em <https://about.gitlab.com>. Depois, faça opcionalmente o [tutorial de uso do próprio GitLab \(em inglês\)](#).

| Trabalhando com o Git

Em primeiro lugar, o Git é um software, feito para operar em linha de comando. É necessário instalá-lo e configurá-lo. Confira os vídeos que acompanham a unidade para isso. Neste tópico, descrevemos os principais conceitos e comandos e porque usá-los, sem entrar em detalhes específicos de instalação ou IDE. Nosso objetivo aqui é que você entenda quais comandos existem, criando um vocabulário básico – que é comum na maior parte dos softwares de controle de versão. Por isso, omitiremos detalhes de alguns comandos mais complexos – que podem ser dados facilmente pela IDE (como o IntelliJ).



EXPERIMENTE

Instalando o Git

Instalando o Git

PUCPR Online



Assista no

Conhecendo o GitHub

Conhecendo o GitHub

PUCPR Online



Assista no

Trabalhando com o Git no Eclipse

Trabalhando com o Git no Eclipse

PUCPR Online



Assista no

Trabalhando com o Git no IntelliJ

Trabalhando com o Git no IntelliJ

PUCPR Online



Assista no

Criando repositórios

Todo controle de versão ocorre em repositórios de código. Para criá-los, utilizamos o comando **git init**. Entretanto, esse comando raramente será usado por você, pois ele criará um repositório somente na máquina local.

A forma mais tradicional de trabalhar é com um repositório remoto, digamos, criado no GitHub. Nesse caso, ao invés de criar um repositório do zero, você gerará um clone e sincronizá-lo de tempos em tempos.

Para fazer isso, usamos o comando **git clone**, seguido do nome do repositório. Isso criará na sua máquina local uma pasta, com a cópia do repositório remoto. Por exemplo, para clonar o repositório do código da JVM do OpenJDK, você poderia usar o comando:

```
git clone https://github.com/openjdk/jdk.git openjdk
```

Isso criaria em sua máquina uma pasta chamada openjdk, com todo o código do repositório remoto dentro. A pasta criada em sua máquina será o **repositório local**. O repositório do Git de onde você clonou é o **repositório remoto**. O nome do repositório (dado no último parâmetro) é opcional e, se omitido, criará a pasta com o mesmo nome do repositório remoto (jdk, neste caso).

Todo código modificado, salvo no disco, compõe ainda um terceiro conceito, chamado **cópia de trabalho**. Isso porque você poderá restaurar um arquivo modificado na sua cópia de trabalho para a versão que está em seu repositório local a qualquer momento.

Mantendo arquivos não controlados

Algumas vezes, você poderá ter arquivos não controlados que deseja manter apenas em seu disco local, mas não os enviar para o repositório. Alguns exemplos de arquivos desse tipo seriam:

- Um arquivo desse tipo poderia ser o xml, em que estão as configurações do banco de dados que a aplicação usará, e que será diferente para cada desenvolvedor da equipe.
- Outra possibilidade seriam os arquivos específicos da sua IDE favorita, que não se referem ao código da aplicação em si.
- Os arquivos binários gerados pela JVM (.class, .java) e que podem ser recompilados por qualquer desenvolvedor.

Para evitar que um arquivo seja controlado, criamos um arquivo chamado *.gitignore* e colocamos lá dentro o nome do arquivo ou pastas que queremos ignorar. Isso evitará que ele seja adicionado caso o *git add* . seja utilizado, que veremos na próxima subseção.

Há, inclusive, uma variação do comando *git rm*, útil para quando queremos remover um arquivo do repositório, mas não do disco – por exemplo, um arquivo que você havia esquecido de adicionar ao *.gitignore*. É por meio da opção *--cached*, por exemplo:

```
git rm --cached README
```

Modificando arquivos no repositório

Todos os arquivos em um repositório podem ser controlados ou não controlados. Os arquivos controlados são aqueles que o Git está monitorando e saberá se foram ou não modificados, ou estão prestes a serem adicionados ao repositório (staged).

Obviamente, você fez o clone de um repositório, todos os arquivos clonados em seu interior serão controlados e não modificados. Quando você modifica um arquivo, ele passará para o estado de arquivo modificado. E, quando você criar um arquivo novo, ele não será controlado. Sua IDE mostrará esses status de maneira visual.

Para que um arquivo seja controlado, é necessário utilizar o comando **git add**. Isso trocará o arquivo criado para um estado chamado de **staged**. Isto é, ele será adicionado ao repositório local assim que as modificações forem confirmadas. Você pode utilizar o comando de duas formas:

- `git add` : para adicionar um único arquivo.
- `git add .` : para adicionar todos os arquivos não controlados.

Para excluir um arquivo, é possível utilizar o comando **git rm**. Ele excluirá o arquivo do disco, mas essa exclusão também será considerada **staged**. Isto é, ela também precisará ser confirmada.

E como confirmamos essas modificações? Por meio do comando **git commit**. Ele não só enviará as informações ao repositório local, como também permitirá que você associe a elas uma mensagem de commit. Trata-se de um texto, no qual você descreve de maneira sucinta as modificações que fez. É esse texto que será exibido caso você queira ver o histórico das suas modificações.

Revertendo alterações

Às vezes, você começou a editar um arquivo e se arrependeu, e gostaria de retorná-lo a versão que está atualmente em seu repositório. Para isso, deve utilizar o comando de **revert**. Ele exigirá o código do último commit, o que pode ser trabalhoso. Felizmente, sua IDE conseguirá obter essa informação automaticamente.

Enviando arquivos ao repositório remoto

Por fim, você pode usar os comandos **git push** e **git pull** para enviar ou receber arquivos do repositório remoto.

Note que você pode trabalhar no repositório local por bastante tempo, antes de dar um push. Isso fará com que o Git agrupe todas as modificações necessárias e as envie de maneira muitíssimo eficiente. Na verdade, sua eficiência foi um dos atributos que o tornou tão popular.

Para onde ir a partir daqui

Obviamente, há muito mais o que estudar sobre o Git. Caso você queira se aprofundar, há também ótimos materiais disponíveis na internet:

- Livro Pro-git, disponível gratuitamente online em: <https://git-scm.com/>.
- Guia prático e simplificado: https://rogerdudler.github.io/git-guide/index.pt_BR.html.

Por fim, saiba que o esforço de aprender o Git **vale a pena**! Não só você terá muito mais tranquilidade ao codificar, como essa é uma ferramenta-padrão na indústria. Você não entrará em uma empresa hoje em dia que não utilize um controle de versão e, muito provavelmente, será o Git.

| Exercícios de fixação



DICA

Não se esqueça de escrever um método **main()** para cada um desses exercícios para que possa testá-los. Também, use a referência da superclasse ou da interface dentro da **main()** ao manipular os objetos para que você possa testar o polimorfismo. No exercício de aplicação dos funcionários, temporários e concursados nós tivemos um exemplo disso: veja que no método **main()** criamos um *ArrayList* de funcionários, que era a superclasse que usamos naquele exemplo.



DICA

Várias vagas de emprego pedem conhecimento em Git. Por isso, opcionalmente sugiro que você faça o seguinte para cada um dos exercícios abaixo:

1. Crie um repositório Git local.
2. Inicie o repositório (git init).
3. Crie um arquivo .gitignore para ignorar a pasta out e arquivos .class.
4. Faça um commit inicial com as classes vazias.
5. Faça um novo commit quando terminar a implementação de cada exercício



EXERCÍCIO

1. Crie uma classe abstrata **ConteudoNetflix** com atributos para **titulo**, **duracao** e um método abstrato **reproduzir()**. Em seguida, crie subclasses como **filme** e **serie** que implementem o método **reproduzir()**.
2. Você está criando um aplicativo com funcionalidades muito parecidas ao Spotify. Defina uma interface **midia** com métodos **play()**, **pause()**, **next()**, **previous()**. Implemente essa interface em classes como **musica** e **podcast**.
3. Crie uma classe abstrata **produto** com atributos **nome**, **preco** e métodos para **getNome()**, **getPreco()**. Agora crie classes concretas como **livro**, **eletronico** que estendem **produto** e adicionam seus próprios atributos e métodos específicos.
4. Defina uma interface **ViagemPagavel** com um método **calcularTarifa()**. Implemente essa interface nas classes **ViagemUberX** e **ViagemUberBlack**, cada uma com suas próprias regras para calcular a tarifa.
5. Você tem um sistema de RH. Para este sistema, crie uma interface **Pagavel** com o método **calcularPagamento()**. Depois, crie uma classe abstrata **Funcionario** com atributos **nome** e **id**. Depois, crie uma classe **FuncionarioClt** que estende **Funcionario** e implementa **Pagavel** (tem salário fixo). Depois, crie uma classe **Freelancer** que não estende **funcionário** (é um prestador externo), mas implementa **Pagavel** (tem valor por hora). Finalmente, no **Main**, crie uma **ArrayList<Pagavel>** e misture funcionários e freelancers, calculando quanto a empresa deve pagar no total.



Exercício 1

```
</>
1  abstract class ConteudoNetflix {
2      protected String titulo;
3      protected double duracao; // em minutos
4
5      public ConteudoNetflix(String titulo, double duracao) {
6          this.titulo = titulo;
7          this.duracao = duracao;
8      }
9
10     public abstract void reproduzir();
11 }
12
13 class Filme extends ConteudoNetflix {
14     public Filme(String titulo, double duracao) {
15         super(titulo, duracao);
16     }
17
18     public void reproduzir() {
19         System.out.println("Reproduzindo o filme: " + this.titulo);
20     }
21 }
```

Exercício 2

```
</>
```

```
1 interface Midia {
2     void play();
3     void pause();
4     void next();
5     void previous();
6 }
7
8 class Musica implements Midia {
9     public void play() {
10         System.out.println("Tocando música...");
11     }
12
13     public void pause() {
14         System.out.println("Música pausada.");
15     }
16
17     public void next() {
18         System.out.println("Tocando próxima música...");
19     }
```

Exercício 3

</>

```
1  abstract class Produto {
2      private String nome;
3      private double preco;
4
5      public Produto(String nome, double preco) {
6          this.nome = nome;
7          this.preco = preco;
8      }
9
10     public String getNome() {
11         return nome;
12     }
13
14     public double getPreco() {
15         return preco;
16     }
17 }
18
19 class Livro extends Produto {
```

Exercício 4

</>

```
1 interface ViagemPagavel {
2     double calcularTarifa();
3 }
4
5 class ViagemUberX implements ViagemPagavel {
6     private double distancia;
7
8     public ViagemUberX(double distancia) {
9         this.distancia = distancia;
10    }
11
12    public double calcularTarifa() {
13        // Supondo que a tarifa do UberX seja de R$1,00 por km.
14        return this.distancia * 1.00;
15    }
16 }
17
18 class ViagemUberBlack implements ViagemPagavel {
19     private double distancia;
```

Exercício 5

</>

```
1 import java.util.ArrayList;
2
3 interface Pagavel {
4     double calcularPagamento();
5 }
6
7 abstract class Funcionario {
8     String nome;
9
10    public Funcionario(String nome) {
11        this.nome = nome;
12    }
13 }
14
15 // classe concreta que herda de uma e implementa outra
16 class FuncionarioClt extends Funcionario implements Pagavel {
17     double salarioMensal;
18
19     public FuncionarioClt(String nome, double salarioMensal) {
```

| Referências

GODOY, V. **Programação orientada a objetos I**. Curitiba: IESDE, 2019.

HORSTMANN, C. S.; CORNELL, G. **Core Java – volume I**. 8. ed. São Paulo: Pearson, 2010.

